ClassGuard — Automated Classroom Exit Monitoring System

Personal Build Guide (Hardware + Firmware + App)

---

1. Design Review — Changes from the Original Concept

Original idea	Why it was changed	What's used instead
UV LED + UV receiver break-beam	UV LEDs are weak, expensive, degrade fast, and stray UV from sunlight/tube lights near a corridor door causes false triggers. Minor eye-safety concern at close range over years of daily use.	**IR break-beam pair** (940nm IR LED + phototransistor) — the same tech used in industrial door counters and elevator safety edges. Cheap, invisible, immune to visible light.
Magnetic wearable device for teachers	A magnet can only trigger an on/off reed switch — it can't identify *which* teacher, and weak magnets/misalignment make it unreliable over time.	**Passive RFID tag** on the same wristband. The reader logs exactly which teacher passed, with no line-of-sight needed.
Identify students by skin colour + height to catch face-covering	**Not implemented — see Section 2 below.**	Height estimate used only as a rough adult-vs-student cross-check; identity confirmation is left to the teacher reviewing the photo, not automated.

---

2. Why Skin-Colour Identification Was Not Built

This deserves its own section since it was explicitly requested and deliberately left out.

It doesn't solve the problem it's aimed at. The break-beam sensor fires the moment someone crosses the doorway threshold — covering a face doesn't stop that detection. The alert already fires regardless of whether the system can identify who it is. Adding face/skin-based ID doesn't make the core system work any better; it just adds a second, much less reliable, system on top.

It doesn't work reliably as engineering. Camera-perceived skin colour shifts drastically with lighting, white balance, and shadow — far more than it varies between two different people standing in the same spot. It would produce constant misidentifications.

It has serious ethical and legal problems. Building a system that sorts children by skin colour is a form of biometric racial profiling, no matter the stated intent, and this kind of system is well documented to perform unevenly across skin tones — meaning it wouldn't even be *fair*, let alone accurate. For a system involving minors, this is a real legal and reputational risk for a school, not just a technical footnote.

What's used instead: - The camera snapshot is evidence for a human (the teacher) to review — not an automated identity match. - A simple **height-band estimate** (short/tall silhouette from the camera frame) is used only to help auto-suppress alerts for obviously-adult figures who forgot their RFID tag, flagged for teacher confirmation rather than acted on automatically. - Teachers already know their own students by classroom context — the system's job is to flag *that* an unauthorized exit happened, not *who* did it.

---

3. System Architecture

```
[Door Frame]
 ├─ IR Break-beam pair (x2, staggered heights) → detects EXIT vs
ENTRY direction
 ├─ RFID reader → detects teacher tag near the door
 ├─ Camera (ESP32-CAM) → snapshot on every beam-break event
 │
 ▼
[Controller: ESP32]  (one per classroom door)
 ├─ Reads sensors, checks local time against cached timetable
 ├─ Sends event over Wi-Fi (MQTT/HTTPS) to central server
 │
 ▼
[Central Server — cloud free-tier or a school PC]
 ├─ Timetable database
 ├─ Decision engine ("is this an allowed exit right now?")
 ├─ Notification service (Email via SMTP, Push via Firebase)
 │
 ▼
[Teacher's Mobile App]
 ├─ View live alerts with photo
 ├─ Set "student out for practical/project" timer override
 ├─ Manage class timetable
```

Core logic: the door controller doesn't decide anything — it just reports "someone exited door 8B at 10:42." The server checks the timetable and any active teacher override before deciding whether to notify. Keeping the decision logic server-side means rules can change without reflashing every door's firmware.

See the diagram in the school document for the physical mounting layout — it applies equally here.

---

4. Budget Plan — Fitting a Prototype into ₹10,000

A single fully-featured demo door (with camera) needs about ₹2,000 in parts. Since you're working with a soldering iron and mains-powered adapters, build this with a parent, teacher, or your school's electronics lab supervising — especially the soldering and mains-power steps.

All prices are India retail, single-unit, approximate (mid-2026).

Core Electronics (One Prototype Door)

Item	Qty	Cost	Line	Best Place to Buy

			Total	
ESP32 Dev Board 30/38-pin (CP2102, WiFi+BT)	1	₹300	₹300	Robocraze.com or Robu.in
ESP32-CAM module (OV2640, WiFi+BT, built-in camera)	1	₹650	₹650	Robu.in
IR break-beam sensor pair (940nm)	2 pairs	₹100	₹200	Robu.in, Robocraze, local electronics market
RC522 RFID reader kit (module + card + keychain tag)	1	₹499	₹499	Flipkart (AptechDeals/Electrobot RFID kit)
Extra RFID keychain tags (more teachers)	3	₹20	₹60	Robu.in, Amazon.in
DS3231 RTC module	1	₹120	₹120	Robu.in, Robocraze
5V 2A USB power adapter + cable	1	₹200	₹200	Local electronics shop, Amazon.in

Electronics subtotal: ≈ ₹2,029

Basic Tools (one-time, reusable for future projects)

Item	Cost	Best Place to Buy
Soldering iron kit (25–40W, stand + solder wire + flux)	₹350–500	Robu.in, Amazon.in, local hardware shop
Digital multimeter (basic)	₹350–450	Robu.in, Amazon.in
Small screwdriver set	₹150	Any hardware shop
Hot glue gun + glue sticks	₹150–200	Amazon.in, local shop
Small breadboard + jumper wire pack	₹200	Robu.in, Robocraze

Tools subtotal: ≈ ₹1,300

Mounting & Enclosure

Item	Cost	Best Place to Buy
Small wall-mountable		Robu.in, local

enclosure box (ABS)	₹150	hardware store
Aluminum L-brackets / mounting tape	₹100	Local hardware store
Cable conduit (1–2m)	₹60	Local electrical shop

Mounting subtotal: ≈ ₹310

Server / App — ₹0 (free tiers)

- Backend hosting: Oracle Cloud free tier or your own laptop for the demo
- Database: Supabase free tier or self-hosted PostgreSQL
- Email alerts: your school's existing Gmail account via the Gmail API
- Push notifications: Firebase free tier
- App build: Flutter/React Native — free, open-source

Total Budget Summary

Category	Cost
Core electronics	₹2,029
Tools (reusable)	₹1,300
Mounting/enclosure	₹310
Server & app software	₹0
Subtotal	**₹3,639**
Contingency (~30%, for a fried ESP32/sensor)	**₹1,100**
Grand Total	**≈ ₹4,700**
Remaining from ₹10,000 budget	**≈ ₹5,300**

Hold onto the ₹5,000+ remainder rather than spending it immediately — it covers a second prototype door (needed to properly test multi-door server logic and direction detection, which a single door can't fully demonstrate), or a nicer 3D-printed enclosure once the design is final. A full school-wide rollout (≈₹1,650–1,900/door) is a separate, later conversation with your school once you have a working demo.

---

5. Build Method — Step by Step

Phase 1: Door Sensor Rig

1. Mount two IR break-beam pairs at the door frame, one at ~1m and one at ~1.5m height, both aimed straight across the doorway gap (see diagram in the school document).
2. Direction logic: whichever beam breaks *first* tells you direction — outer beam first = exiting, inner beam first = entering.
3. Debounce in firmware: ignore re-triggers within 300ms to prevent double-counting one person.

Phase 2: Teacher ID Module

1. Mount the RC522 RFID reader on the frame at wrist height, corridor side.
2. Program each teacher's RFID tag with a unique ID mapped to their name in the database.
3. Firmware logic: if the reader detects a registered teacher tag within ~1 second of a beam-break event, tag the event "teacher passage" and suppress the notification.

Phase 3: Camera

1. Use ESP32-CAM (camera + controller combined) to save wiring.
2. On every beam-break event, capture a JPEG snapshot, timestamp it, and attach it to the event for the teacher to review later. This also deters false alarms and pranks.
3. Height-band estimate (optional, Phase 2 feature): estimate whether the figure in frame is adult-height or student-height using the fraction of the frame height the silhouette occupies — a rough signal only, always paired with the photo for human confirmation, never used alone to identify a specific person.

Phase 4: Controller Firmware (ESP32, Arduino IDE or PlatformIO, C++)

```
loop():
  read IR beam states
  if beam sequence = EXIT pattern:
      check RFID reader status
      if teacher_tag_detected: log "teacher_exit", skip alert
      else:
          capture camera snapshot
          send event {door_id, timestamp, type:"student_exit",
image}
          → via MQTT/HTTPS to server
```

The controller stays “dumb” about rules — timetable logic lives on the server so rules can be updated centrally without reflashing every door.

Phase 5: Backend Server

1. Database tables: `Timetable(class_id, day, period, start_time, end_time, is_break)`, `Teachers(id, name, rfid_tag, phone, email)`, `Events(door_id, timestamp, type, image_url)`, `Overrides(class_id, student_name, start_time, end_time, reason)`.
2. On a `student_exit` event: check if the current period `is_break` → no alert. Else check `Overrides` for an active teacher-set timer covering this door/time → no alert until it expires. Else → notify class teacher by email + push notification to nearby on-duty teachers.
3. When an override timer expires, push a “time’s up, not yet back” reminder if no matching `entry` event was logged.

Phase 6: Mobile App (Teacher-Facing)

Build with Flutter or React Native, backend via REST API + Firebase Cloud Messaging: - **Login** (teacher credentials) - **Live Alerts tab**: real-time exit events with snapshot, door, time - **Timetable Manager**: upload/edit class timetable - **Override/Timer tool**: select class + door + duration → creates an `Overrides` entry, auto-expires and notifies - **Teacher Registry**: admin screen to register RFID tag ↔ teacher identity

---

6. Uses & Applications

Direct school benefits

- Reduces class-skipping — real-time detection instead of relying on manual roll-calls.
- Attendance accuracy — factual exit/entry logs reduce disputes about who left and when.
- Faster emergency headcounts — during a fire drill or real emergency, the system already knows who exited each room and when.
- Reduces teacher workload — teachers are alerted only when something is actually wrong, not asked to watch the door every period.

- Flags unauthorized entry too — an unrecognized person entering during a restricted period (e.g. an outsider during an exam) triggers the same alert path.
- Supports counseling, not just punishment — a pattern of repeated unauthorized exits by the same student can flag someone who may need support rather than just discipline.

Extended uses within the school

- Exam halls: flag any student leaving during a restricted window without a proctor's override.
- Labs/library: monitor entry-exit for expensive equipment areas or after-hours access.
- Hostel/dormitory doors: curfew monitoring using the same hardware, minus the timetable logic.

Beyond the classroom

- Strong science-fair/robotics-competition/college-application project — demonstrates embedded systems, IoT networking, backend design, and mobile development in one build.
- Skills transfer directly to future IoT projects: smart attendance systems, home automation, asset tracking.
- Publishing the firmware and backend code on GitHub once working could help other students or schools, and is a solid addition to a resume or application.

---

7. Key Pitfalls to Design Around

- **Two people passing together**: a break-beam pair can't distinguish 1 vs 2 people. The camera snapshot is the backup — a human reviews flagged events rather than the sensor being trusted blindly.
- **Wi-Fi dependency**: buffer events locally on the ESP32 and retry sending if Wi-Fi drops, so no exit goes unlogged.
- **Power cuts**: the DS3231 RTC module keeps correct time even after a reboot, so the timetable check stays accurate.
- **Tampering**: mount sensors at height, out of easy reach, and log "sensor obstructed continuously" as its own alert type.
- **Privacy**: since this captures photos of minors, get institutional approval, restrict image access, set a short retention period, and inform parents per your school's data-protection policy before deployment — not as an afterthought.

---

8. Suggested Build Order (Timeline)

1. **Week 1–2**: Prototype single door on breadboard (IR beam + RFID + ESP32), test locally with serial monitor.
2. **Week 3**: Add camera, get MQTT working to a test server.
3. **Week 4**: Build minimal backend (timetable + notification logic), test end-to-end with one door.
4. **Week 5–6**: Build the mobile app's alert + override screens.
5. **Week 7**: Finalize enclosure, mount on a real classroom door, pilot test for a week.
6. **Week 8+**: If approved, roll out to more doors and tune false-positive handling using real data.